Neural networks for decoding error correcting codes in data communications

Presentation by Fatemeh Jalili

Deep Learning | Prof. Yousefi

11 February, 2026

Agenda

1. System model & LLRs
2. Baseline decoding
3. Neural decoders
4. Learned BP
5. Adaptive learned BP
6. GNN decoder

System model

- Linear block code $\mathcal{C} \subset \{0,1\}^n$, dimension k, rate $r = k/n$
- Parity check: $Hc^T = 0 \ (\text{in GF}(2))$
- BPSK over AWGN:

$$y_i = (-1)^{c_i} A + z_i, \quad z_i \sim \mathcal{N}(0, \sigma^2)$$

- Channel LLR:

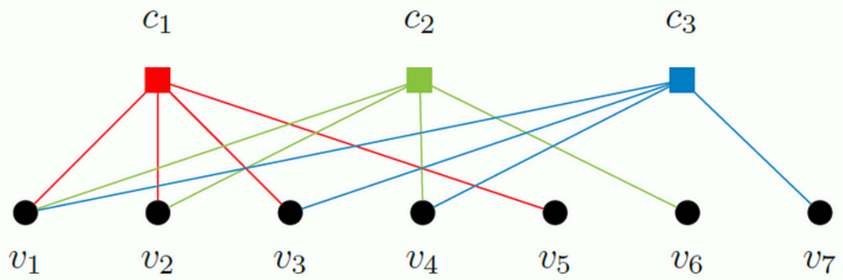
$$L_i \triangleq \log \frac{p(c_i = 0 \mid y_i)}{p(c_i = 1 \mid y_i)} \propto \frac{2y_i}{\sigma^2}$$

Tanner graph + standard BP / min-sum

- Tanner graph: variable nodes $v \in V$ (bits), check nodes $c \in C$ (parity constraints), edges E
- Neighborhoods: $C_v = \{c : (c, v) \in E\}$, $V_c = \{v : (c, v) \in E\}$
- Messages (iteration t):
  - Variable → check: $L^{(t)}_{v \to c}$
  - Check → variable: $L^{(t)}_{c \to v}$

$$\mathbf{H} = \begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

(a) Parity-check matrix.



(b) Tanner graph.

Fig. 1: Bipartite graph for the (7,4) Hamming code.

Tanner graph + standard BP / min-sum

- Sum-product (check update):

$$L_{c\to v}^{(t)} = 2\,\tanh^{-1}\left(\prod_{v'\in V_c\setminus\{v\}} \tanh\left(\frac{L_{v'\to c}^{(t-1)}}{2}\right)\right)$$

- Variable update:

$$L_{v\to c}^{(t)} = L_v^{\mathrm{ch}} + \sum_{c'\in C_v\setminus\{c\}} L_{c'\to v}^{(t)}$$

- Min-sum approximation (check):

$$L_{c\to v}^{(t)} \approx \left(\prod \mathrm{sign}(\cdot)\right) \min_{v'\neq v} \left|L_{v'\to c}^{(t-1)}\right|$$

Neural Decoders

Model-agnostic

- MLP / CNN / RNN / Autoencoders / Transformers / GNNs

Model-based

- Unrolled BP / min-sum + trainable weights
- Weighted BP (WBP), neural normalized min-sum, etc.

Weighted Belief Propagation (WBP) / Weighted Min-Sum (WMS)

- Check update (sum-product form + min-sum approx):

$$L_{c\to v}^{(t)} = 2\tanh^{-1}\left(\prod_{v'\in V_c\setminus\{v\}} \tanh\left(\frac{\gamma_{v',c}^{(t)}}{2} L_{v'\to c}^{(t-1)}\right)\right)$$

$$\approx \underbrace{\min_{v'\neq v}\left|L_{v'\to c}^{(t-1)}\right|}_{\bar{L}_{c,v}} \prod_{v'\neq v} \gamma_{v',c}^{(t)} \operatorname{sign}\left(L_{v'\to c}^{(t-1)}\right)$$

- Variable update:

$$L_{v\to c}^{(t)} = \alpha_v^{(t)} L_v^{\mathrm{ch}} + \sum_{c'\in C_v\setminus\{c\}} \beta_{c',v}^{(t)} L_{c'\to v}^{(t-1)}$$

- Decision / syndrome stopping:

$$\hat{c}_v^{(t)} = \mathbf{1}\{L_v^{(t)} < 0\}, \quad \boldsymbol{s} = \hat{\boldsymbol{c}}^{(t)} H^T$$

Learned BP

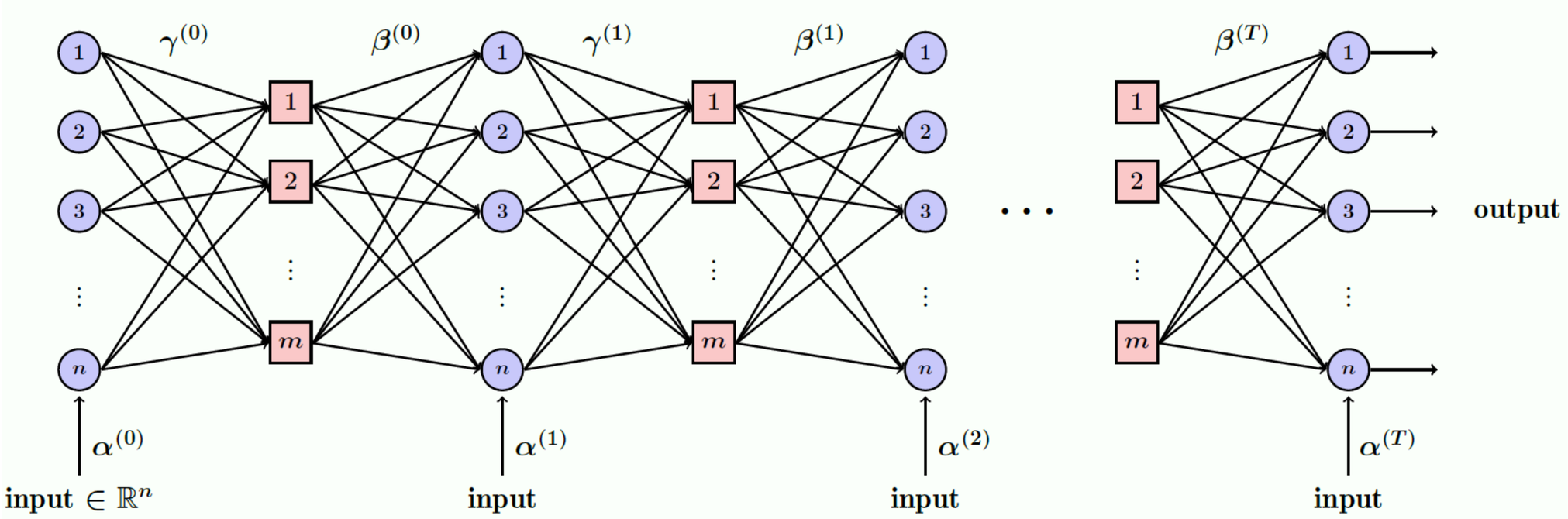


Fig. 2: Tanner graph unrolled to an RNN.

Parameter sharing

- Without sharing: weights per edge and per iteration → many parameters
- Sharing schemes (examples):
  - **Type Ta:** $\gamma_{v,c}^{(t)} = \beta_{c,v}^{(t)} = \gamma_{v,c}$ (shared across t)
  - **Type TaVC:** single global γ (very small parameter count)
  - **Type TbVC:** one $\gamma^{(t)}$ per iteration
- Trade-off: fewer parameters ↔ less flexibility

Adaptive Learned BP

- Static WBP learns fixed weights offline; adaptive WBP learns a **function**

$$\gamma = g(y) \in \mathbb{R}_+^T, \quad \min_g \Pr\left(D_{g(y)}(y) \neq c\right)$$

Parallel adaptive decoder

- **Idea:** adapt weights per received word by trying a small set of candidates $\gamma = (\gamma^{(1)}, \ldots, \gamma^{(T)}) \in \mathbb{R}_+^T$
- Define a discrete candidate set $X = \{\gamma^1, \ldots, \gamma^\nu\}$
- Run ν decoders in parallel:

$$\hat{c}_i = D_{\gamma^i}(y), \quad i = 1, \ldots, \nu$$

- Select best candidate using **syndrome weight**:

$$s_i = \hat{c}_i H^T, \qquad i^* = \arg\min_i \|s_i\|_H, \qquad \hat{c} = \hat{c}_{i^*}$$

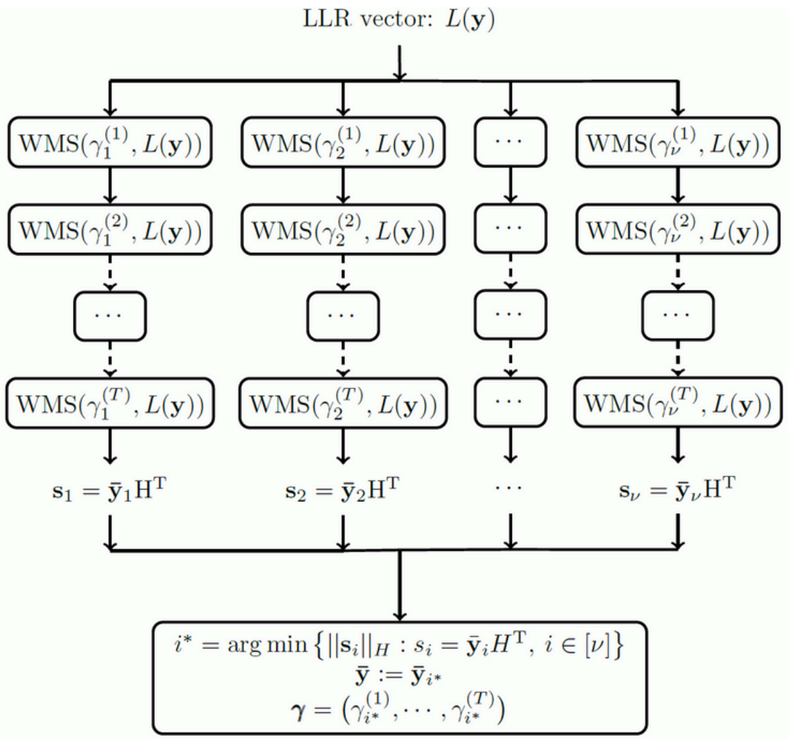


Fig. 3: Adaptive decoders.

Two-stage decoder

- Replace online search with a lightweight predictor:

$$\bar{\gamma} = g_\theta(y) \in \mathbb{R}_+^T, \qquad \hat{c} = D_{\bar{\gamma}}(y)$$

- Train g_θ to imitate "good" weights (targets from offline procedure)
- Paper uses a small CNN + **quantile loss**:

$$\ell_\xi(\gamma, \bar{\gamma}) = \mathrm{mean}\Big(\max(\xi(\gamma - \bar{\gamma}), (\xi - 1)(\gamma - \bar{\gamma})) \Big)$$

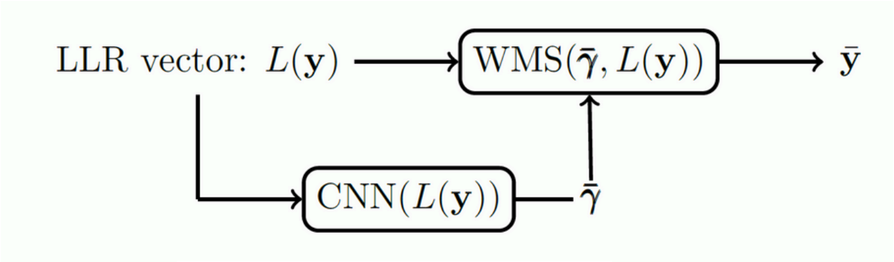


Fig. 3: Adaptive decoders.

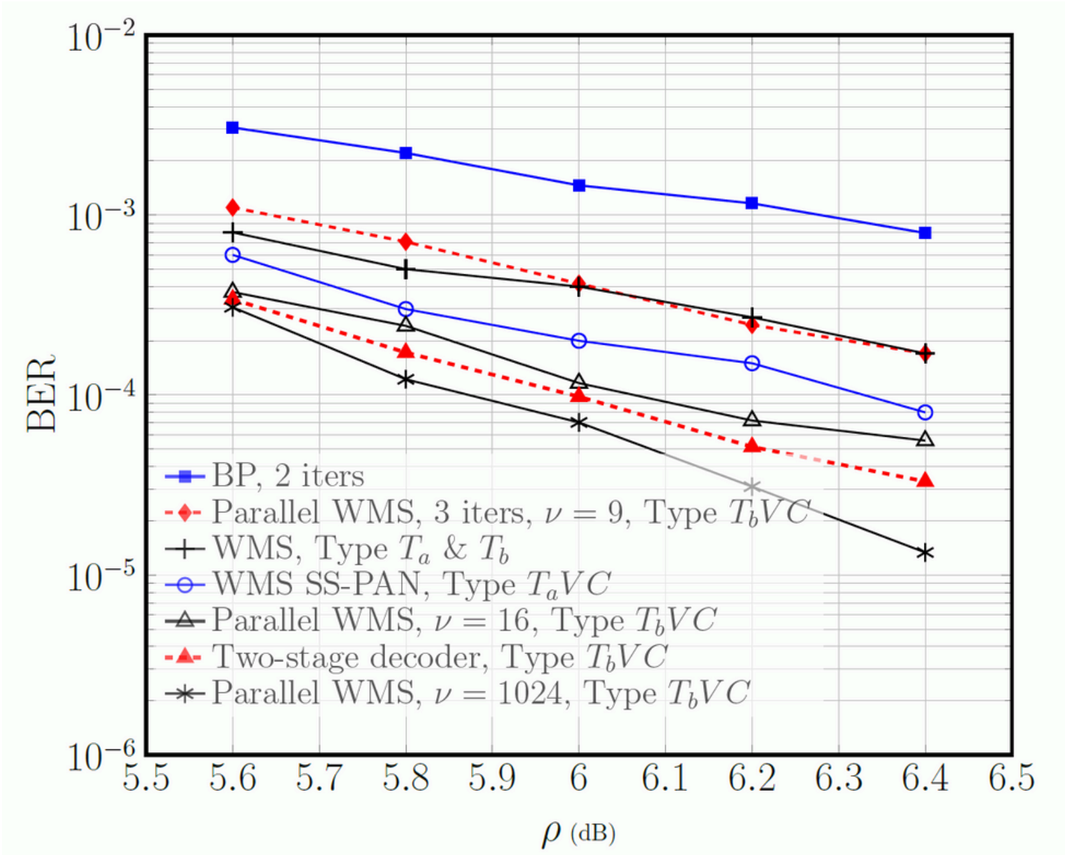


Fig. 4: BER versus SNR ρ, for the AWGN channel in the low-rate regime.

GNN decoder

- Represent code by Tanner graph from H
- Replace BP update rules by **trainable functions** (MLPs)
- Bipartite message passing (Cammerer et al.):

$$m_{v_i \to u_j} = f\left(\left[h_{v_i} \| h_{u_j} \| g_{v_i \to u_j} \right] ; \theta_{v \to u} \right)$$

$$m_{u_j \to v_i} = f\left(\left[h_{u_j} \| h_{v_i} \| g_{u_j \to v_i} \right] ; \theta_{u \to v} \right)$$

$$h'_{v_i} = f\left(\left[h_{v_i} \| \mathrm{Agg}(\{ m_{u \to v_i} \}) \| g_{v_i} \right] ; \theta_v \right)$$

$$h'_{u_j} = f\left(\left[h_{u_j} \| \mathrm{Agg}(\{ m_{v \to u_j} \}) \| g_{u_j} \right] ; \theta_u \right)$$

- Initialization from channel LLRs; iterate N_{iter} times

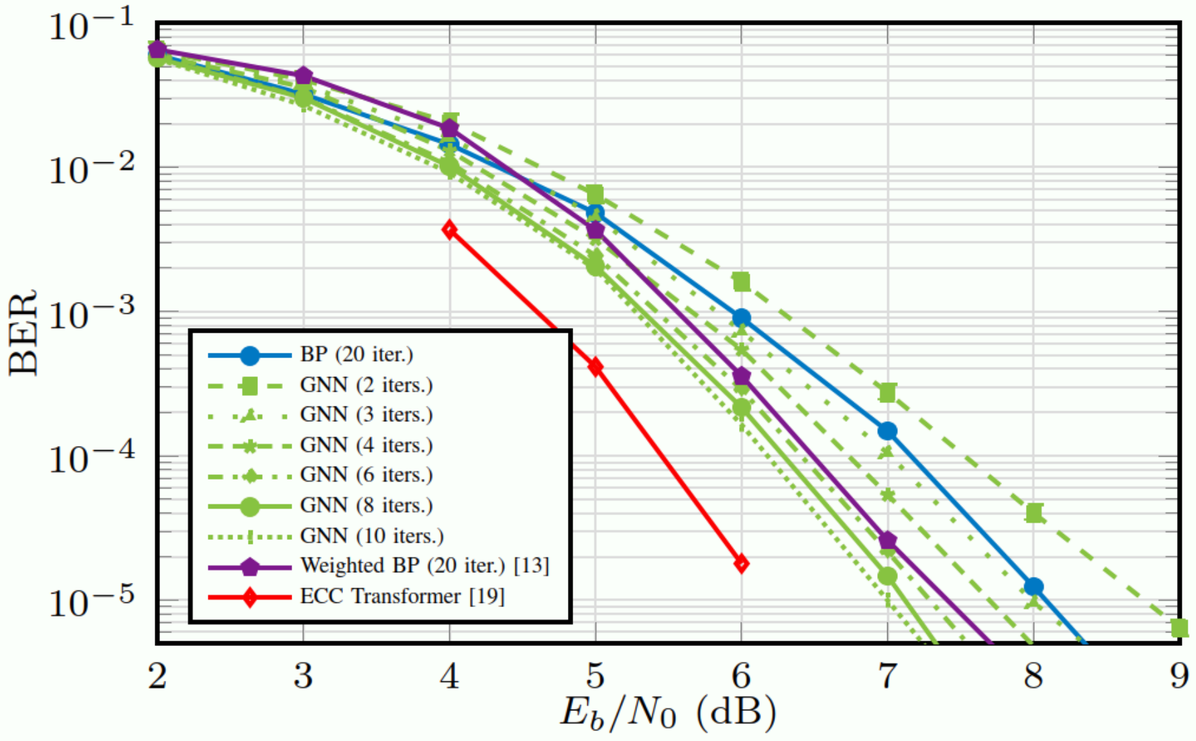


Fig. 5: BER performance for decoding of the (63,45) BCH code. The GNN was trained for $N_{\text{iter}} = 8$ iterations.

Conclusion